

Backend Internship Task — Complete VELoop Rewards Wallet & Payout System

Project: VELoop Rewards — Wallet & Withdrawal Backend System

Task Type: Backend-Focused Full-Stack Demonstration Project

Primary Focus: Backend Architecture, Wallet Ledger, Transactions, Withdrawals & Security

Frontend: Simple demonstration frontend only

Technology: MERN / Node.js + Express.js + MongoDB + React.js

Reference Pages

- **Wallet:** [VELoop Rewards Wallet](#)
- **Payout:** VELoop Rewards Payout

Important: The goal is to build a real, backend-driven wallet system for VELoop Rewards—not a static UI or a frontend-only balance simulator. The existing Wallet and Payout pages should be used as the functional reference. The demonstration frontend should expose the actual backend data and workflows.

1. Project Objective

VELoop Rewards uses multiple internal reward currencies and redemption mechanisms.

The objective of this task is to develop a **complete, secure and scalable wallet backend system** that can manage:

- User wallet balances
- VEs
- SVEs
- Gems
- Tokens

- Spins
- Wallet transactions
- Earnings
- Deductions
- Exchange-related balance changes
- Withdrawal requests
- Withdrawal status
- Payout methods
- Payout options/vouchers
- User payout details
- Transaction history
- Balance validation
- Withdrawal eligibility
- Fraud/security checks
- Audit records

The system must be designed so that **the backend is always the source of truth**.

A user must never be able to simply modify:

VEs: 1000 → 100000

SVEs: 500 → 50000

from the browser and receive the modified balance.

2. Understand the Existing VELoop Wallet

Before starting development, carefully inspect the existing:

/Wallet

page and understand what information is actually displayed to users.

The current platform documentation describes the Wallet & Exchange area as dealing with **VEs, Gems, Tokens and Spins**, while VEs are used for supported redemption requests.

Important restriction

Do not invent additional wallet sections just to make the project look bigger.

The demonstration wallet should contain the information and concepts actually relevant to the current VELoop Rewards Wallet.

If a particular value/section is not currently shown on /Wallet, don't create an unrelated new wallet feature unless it is required internally for the backend.

3. Core Requirement — Backend-Driven Wallet

The entire wallet must work from the backend.

Example

The frontend requests:

GET /api/wallet

The backend returns the authenticated user's wallet information.

Example structure:

```
{
  "success": true,
  "wallet": {
    "ves": 12500,
    "sves": 4200,
    "gems": 85,
    "tokens": 350,
    "spins": 4
  }
}
```

The exact response structure may be improved by the intern.

Never do this

```
const balance = 12500;
```

or:

```
localStorage.setItem("ves", "12500");
```

as the actual wallet source.

LocalStorage may be used for non-sensitive UI preferences, but **wallet balances must come from the backend.**

4. Wallet Database Design

Create a proper wallet architecture.

You may use separate collections or a carefully designed wallet model.

A recommended structure is:

User

```
User
├── userId
├── email
├── name
├── accountStatus
└── wallet reference
```

Wallet

```
Wallet
├── userId
├── VEs
├── SVEs
├── Gems
├── Tokens
├── Spins
├── createdAt
└── updatedAt
```

The intern is free to improve this structure if there is a better production architecture.

5. Wallet Ledger — VERY IMPORTANT

Do **not** rely only on a current balance.

Every wallet change should have a transaction/ledger record.

For example:

Transaction

- transactionId
- userId
- currency
- type
- amount
- balanceBefore
- balanceAfter
- source
- referenceId
- status
- description
- metadata
- createdAt
- updatedAt

Example

A user earns:

+500 VEs

Ledger:

Currency: VEs

Type: CREDIT

Amount: 500

Balance Before: 10,000

Balance After: 10,500

Source: WATCH_AD

If a user withdraws:

-2,400 VEs

the ledger should record:

Currency: VEs

Type: DEBIT

Amount: 2400

Balance Before: 10,500

Balance After: 8,100

Source: WITHDRAWAL

Reference: withdrawalId

This gives VELoop an auditable wallet history.

6. Transaction Types

Implement a proper transaction-type system.

Examples:

Credit

REWARD

BONUS

REFERRAL

DAILY_REWARD

AD_REWARD

GAME_REWARD

ADMIN_CREDIT

EXCHANGE_CREDIT

Debit

WITHDRAWAL

EXCHANGE_DEBIT

ADMIN_DEBIT

CORRECTION

You may add additional types when they are genuinely required.

7. Atomic Wallet Operations

Wallet operations must be safe against race conditions.

For example, a user has:

10,000 VEs

and sends two withdrawal requests simultaneously.

The system must **not** allow both requests to successfully deduct the same balance.

Use appropriate MongoDB atomic operations/transactions where applicable.

The following must happen as one logically consistent operation:

```
graph TD
    A[Check balance] --> B[Validate withdrawal]
    B --> C[Deduct wallet]
    C --> D[Create ledger entry]
    D --> E[Create withdrawal]
```

If one critical step fails, the wallet should not end up partially modified.

8. Balance Validation

Every debit operation must verify the available balance on the server.

Example:

Required: 2,400 VEs
User balance: 1,900 VEs

The request must be rejected.

Example response:

```
{
  "success": false,
  "message": "Insufficient VEs balance."
}
```

Never trust:

amount
balance
currency

withdrawal value

coming from the frontend without server-side validation.

9. Wallet API Requirements

Create clean REST APIs.

Recommended APIs:

Get Wallet

GET /api/wallet

Returns the authenticated user's current wallet.

Get Wallet Transactions

GET /api/wallet/transactions

Should support pagination.

Example:

GET /api/wallet/transactions?page=1&limit=20

Get Wallet Summary

GET /api/wallet/summary

Can return the data needed by the Wallet dashboard.

Credit Wallet

Internal/protected API:

POST /api/wallet/credit

This should **not** be publicly accessible to normal users.

Debit Wallet

Internal/protected API:

POST /api/wallet/debit

Again, this must be protected.

10. Internal Wallet Service

Do not put all wallet logic directly inside route files.

Create a service such as:

walletService.js

Responsibilities:

creditWallet()
debitWallet()
getWallet()
getTransactions()
validateBalance()
createLedgerEntry()

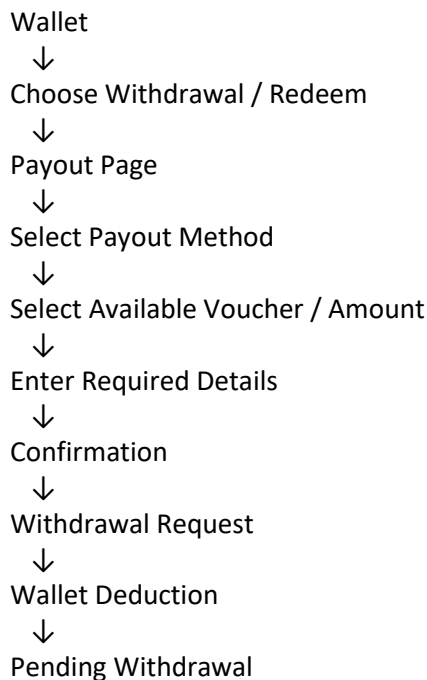
This makes the wallet reusable by:

- Watch Ads
 - Referrals
 - Daily Rewards
 - Games
 - Spin
 - Mining
 - Exchanges
 - Withdrawals
 - Admin operations
-

11. Withdrawal / Payout System

When the user chooses the withdrawal option from Wallet, the system should take them to the **Payout page**.

The flow should be:



The platform currently documents supported payout methods including **UPI, PayPal, Amazon Gift Cards and Google Play Gift Cards**, subject to eligibility and platform availability.

Important

For this task, the intern must inspect the actual current Payout UI and implement the options relevant to the current platform rather than inventing unrelated payout products.

12. Payout Options Must Come From Backend

Do not hardcode the payout options in React.

Bad:

```
const payoutOptions = [  
  { amount: 10 },  
  { amount: 25 },  
  { amount: 50 }  
];
```

Instead:

GET /api/payout/options

Example:

```
{  
  "success": true,  
  "methods": [  
    {  
      "method": "upi",  
      "name": "UPI",  
      "options": []  
    },  
    {  
      "method": "amazon",  
      "name": "Amazon Gift Card",  
      "options": []  
    },  
    {  
      "method": "google_play",  
      "name": "Google Play Gift Card",  
      "options": []  
    }  
  ]  
}
```

The backend should control:

- payout method
 - voucher
 - denomination
 - required currency
 - required balance
 - availability
 - eligibility
 - status
-

13. Payout Values

Use the **currently configured VELoop payout values** as the reference.

The previously configured UPI redemption structure includes:

Redemption VEs Required

₹10	2,400 VEs
₹25	5,800 VEs
₹50	10,000 VEs
₹100	19,500 VEs
₹150	28,500 VEs
₹300	52,500 VEs
₹500	80,500 VEs
₹1,000	150,000 VEs

Do not put these values directly into frontend business logic.

They should be seeded/configured in the backend/database.

If the live Payout page has a different currently displayed configuration, the intern must follow the **current platform configuration** rather than blindly using this table.

14. Payout Method Details

When a user selects a payout method, the required information should be collected according to that method.

UPI

Example:

UPI ID

Gift Card

For gift-card redemption, collect the required user information according to the current VELoop payout flow.

The backend must validate the submitted data.

15. Withdrawal Model

Create a dedicated:

Withdrawal

collection/model.

Recommended structure:

Withdrawal

- withdrawalId
- userId
- method
- optionId
- currency
- currencyAmount
- payoutAmount
- payoutDetails
- status
- rejectionReason
- reviewNote
- transactionId
- requestedAt
- processedAt
- updatedAt

16. Withdrawal Status System

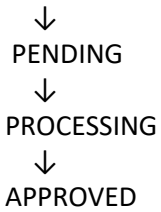
Implement:

PENDING
PROCESSING
APPROVED
REJECTED
CANCELLED

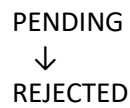
The exact state flow may be improved if required.

Example:

User requests withdrawal



or:

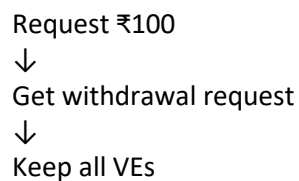


The system must preserve the withdrawal history.

17. Wallet Deduction & Withdrawal Safety

This is one of the most important parts of the task.

A user must not be able to:



The system needs a clear financial-state strategy.

The intern must decide and document whether the platform:

Option A

Deducts the balance immediately when the withdrawal is created.

or

Option B

Places the amount into a reserved/held balance until the withdrawal is processed.

Either approach is acceptable **if implemented consistently and safely**.

For example:

Available VEs
10,000

Withdrawal
2,400

Available
7,600

If the request is rejected, the system must correctly handle the reversal according to the chosen architecture.

18. Idempotency

Duplicate requests must be prevented.

For example, a user double-clicks:

Redeem
Redeem

within milliseconds.

The system must not create:

2 withdrawals
2 wallet deductions

for one intended request.

Implement an appropriate idempotency strategy.

19. Authentication & Authorization

All wallet APIs must require authentication.

Example:

JWT
↓
Authentication Middleware
↓
Wallet Controller

The backend must identify the user from the authenticated session/token.

Never allow:

GET /api/wallet?userId=someoneElse

to expose another user's wallet.

20. Security Requirements

Implement reasonable production-level protections including:

Authentication

- JWT/session validation
- Protected routes
- Token verification

Authorization

- User-level access control
- Admin/internal operations protected

Validation

Validate:

- currency
- amount
- payout method
- payout option
- payout details

- user status

Rate limiting

Protect sensitive endpoints such as:

withdrawal
wallet mutations
OTP/authentication

Duplicate request protection

Prevent repeated submissions.

Input sanitization

Do not trust client input.

21. Do Not Trust Frontend Data

The following should always be calculated/verified on the backend:

Current balance
Required VEs
Payout value
Eligibility
Withdrawal amount
Transaction status
Reward amount
Available payout options

For example, the frontend may send:

```
{  
  "method": "upi",  
  "optionId": "upi_100"  
}
```

The backend should look up:

upi_100

and determine the actual required VEs.

The frontend must not send:

```
{  
  "amount": 19500  
}
```

and expect the backend to trust it.

22. Payout Configuration System

Create a backend configuration structure for payout options.

Example:

```
PayoutMethod  
├── methodId  
├── name  
├── type  
├── currency  
├── payoutValue  
├── requiredAmount  
├── active  
├── eligibility  
└── metadata
```

This allows VELoop to change:

₹100 → 19,500 VEs

without rebuilding the React frontend.

23. Transaction History

The Wallet demonstration should show the wallet transaction data that is actually relevant to the Wallet experience.

Examples:

+500 VEs
Watch Ad Reward

+100 VEs
Daily Reward

-2,400 VEs
UPI Withdrawal

+20 Gems
Reward

Each record should ideally have:

Type
Amount
Currency
Description
Status
Date

Use pagination rather than loading thousands of records.

24. Wallet Summary

The demonstration frontend should display the relevant wallet values from the backend.

For example:

VELoop Wallet

VEs
12,500

SVEs
4,200

Gems
85

Tokens
350

Spins
4

Only include values actually represented in the current Wallet experience.

The objective is to reproduce the platform's real wallet data model, not create an unrelated dashboard.

25. Simple Frontend Demonstration

Although this is a **backend-focused task**, a small React frontend is mandatory.

Its purpose is to prove that the backend actually works.

Required pages/routes

/wallet
/payout

You may add:

/login

or a simple demo authentication mechanism if required.

26. Demonstration Frontend — Wallet

The frontend should:

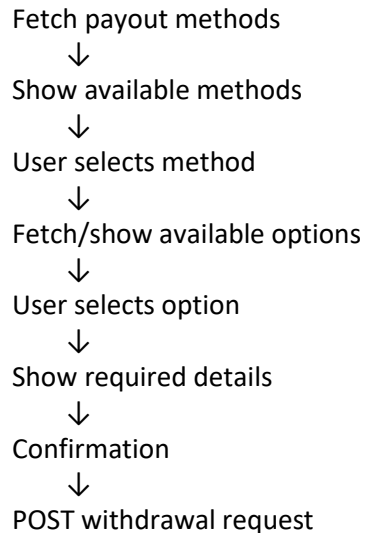
1. Fetch wallet data from backend.
2. Display current balances.
3. Display relevant transaction information.
4. Provide the withdrawal/redeem action.
5. Navigate to the Payout page.
6. Show loading state.
7. Show error state.
8. Refresh wallet data after successful operations.

Do not spend most of the task time creating a beautiful frontend.

Backend quality is the primary evaluation criterion.

27. Demonstration Frontend — Payout

The Payout page should:



The data must come from the backend.

28. Withdrawal Confirmation

Before creating a withdrawal:

Are you sure you want to redeem this reward?

Payout: ₹100

Required: 19,500 VEs

[Cancel] [Confirm]

The backend must still perform the final validation.

A confirmation modal is **not a security mechanism**.

29. After Withdrawal

After successful withdrawal:

Withdrawal submitted successfully.

Status: Pending

The wallet should refresh.

Example:

Before:

VEs
25,000

Withdrawal
₹100
19,500 VEs

After:

VEs
5,500

The transaction history should contain the corresponding debit.

30. Error Handling

Implement proper backend errors.

Examples:

Insufficient balance

Insufficient VEs balance.

Invalid payout option

Selected payout option is unavailable.

Inactive payout method

This payout method is currently unavailable.

Duplicate withdrawal

This withdrawal request has already been submitted.

Authentication

Authentication required.

Suspicious activity

This request requires additional verification.

Don't expose internal database errors to users.

31. Admin/Internal Control

You do **not** need to build the entire VELoop Admin Panel.

However, create the backend architecture so that an administrator/internal service could later:

- View withdrawal
- Approve withdrawal
- Reject withdrawal
- Add wallet credit
- Deduct wallet balance
- Review transactions

Any administrative wallet modification must be:

- Authenticated
- Authorized
- Logged
- Auditable

32. Audit Logs

Create an audit mechanism for sensitive wallet actions.

For example:

```
AuditLog
├── actorId
├── action
├── targetUserId
└── targetType
```

- referenceId
- metadata
- IP/session information where appropriate
- createdAt

Important events:

- Withdrawal Created
- Withdrawal Approved
- Withdrawal Rejected
- Wallet Credit
- Wallet Debit
- Balance Correction
- Payout Configuration Changed

This becomes extremely useful when investigating disputes.

33. Database Indexing

Consider appropriate indexes for:

- userId
- transactionId
- withdrawalId
- referenceId
- createdAt
- status

The goal is to keep wallet and withdrawal queries efficient as the platform grows.

34. Data Consistency Requirement

At the end of the task, this must always be explainable:

Current Wallet Balance
=
Initial Balance
+ Credits
- Debits
± Adjustments

The ledger should allow developers to investigate how the current balance was reached.

35. Important Real-World Scenario Testing

The intern must test at least these cases:

Test 1 — Normal credit

1000 VEs
+
500 VEs
=
1500 VEs

Test 2 — Normal withdrawal

1500 VEs
-
1000 VEs
=
500 VEs

Test 3 — Insufficient balance

500 VEs
withdraw 1000 VEs
→ rejected

Test 4 — Double-click withdrawal

Two identical requests
→ only one succeeds

Test 5 — Concurrent withdrawals

Balance: 10,000 VEs

Request A: 8,000
Request B: 8,000

→ cannot both succeed

Test 6 — Invalid payout option

Fake option ID
→ rejected

Test 7 — Another user's wallet

User A attempts User B wallet
→ rejected

Test 8 — Rejected withdrawal

Verify that the chosen balance strategy correctly handles the reversal/release.

Test 9 — API manipulation

Modify frontend request values.

Backend must still calculate/validate the actual amount.

36. API Documentation

Create:

API_DOCUMENTATION.md

Include:

Endpoint
Method
Authentication
Request
Response
Errors
Example

Document all major APIs.

Example:

GET /api/wallet
GET /api/wallet/transactions
GET /api/payout/methods
GET /api/payout/options/:method

POST /api/withdrawals
GET /api/withdrawals
GET /api/withdrawals/:id

37. Environment Variables

Sensitive values must not be committed to GitHub.

Use:

```
PORT=  
MONGO_URI=  
JWT_SECRET=
```

and any other required configuration.

Commit:

```
.env.example
```

but **never**:

```
.env
```

with real credentials.

38. Suggested Backend Structure

A clean structure could look like:

```
backend/  
|  
|├── src/  
|│   ├── config/  
|│   ├── controllers/  
|│   ├── middleware/  
|│   ├── models/  
|│   ├── routes/  
|│   ├── services/  
|│   ├── utils/  
|│   ├── validators/  
|│   └── migrations/  
|└── app.js
```

```
|
|├── seed/
|├── payoutOptions.js
|
|├── .env.example
|├── package.json
|└── README.md
```

The intern is free to use a better architecture.

39. Recommended Models

At minimum:

User
Wallet
WalletTransaction
Withdrawal
PayoutOption
AuditLog

You may combine or separate models where architecturally appropriate.

40. Seed Data

Create realistic demonstration data.

Example user:

User:
demo@veloop.test

Example wallet:

VEs: 25,000
SVEs: 5,000
Gems: 100
Tokens: 500
Spins: 3

Create sample transactions such as:

Ad Reward
Daily Reward
Referral Reward
Bonus
Withdrawal

Use clearly marked **demo/test data** rather than pretending it is production user data.

41. Do Not Connect to Production

For this internship task:

Do NOT give the intern production database credentials.

They must create and use their own:

MongoDB database

and development environment.

The demonstration should work completely on their own local/development backend.

This makes the task safer while still testing whether they can build a genuine backend system.

42. Production-Like Architecture Without Production Access

The goal is:

Intern's Local Frontend



Intern's Local Backend



Intern's MongoDB

not:

Intern



VELoop Production Database

After review, selected backend code can be integrated into the actual VELoop codebase by the authorized development team.

43. README Requirements

The README must explain:

Project

VELoop Rewards Wallet System

Technologies

Node.js

Express.js

MongoDB

Mongoose

React

Bootstrap/CSS

JWT

Setup

git clone

npm install

environment setup

database setup

npm run dev

API Documentation

Explain major endpoints.

Database Architecture

Explain:

Wallet

Ledger

Withdrawal

Payout
Audit

Security

Explain:

Authentication
Authorization
Validation
Atomic operations
Idempotency
Rate limiting

Testing

Explain how the evaluator can test:

Credit
Debit
Withdrawal
Insufficient balance
Duplicate requests
Payout selection

44. GitHub Requirements

The complete project must be pushed to GitHub.

Repository should contain:

/backend
/frontend
README.md
API_DOCUMENTATION.md
.env.example

Do not commit:

.env
credentials
passwords
MongoDB connection strings
JWT secrets
API keys

45. Deployment

The intern should deploy the demonstration project if possible.

Recommended:

Frontend

Vercel / Netlify

Backend

Render / Railway / another suitable platform

Database

MongoDB Atlas

Use only the intern's own demonstration database.

46. What Will Be Evaluated Most

This is **not primarily a UI-design task**.

Evaluation priority:

1. Backend architecture — 20%

Is the system structured professionally?

2. Wallet correctness — 20%

Do balances and transactions remain accurate?

3. Security — 15%

Can the frontend manipulate balances?

Can one user access another user's wallet?

4. Withdrawal system — 15%

Does the payout flow work correctly?

5. Database design — 10%

Are wallet, ledger and withdrawal records properly structured?

6. Edge cases — 10%

Does the system survive duplicate/concurrent requests?

7. API quality — 5%

Are APIs clean and documented?

8. Demonstration frontend — 5%

Does it properly prove the backend works?

47. Final Deliverables

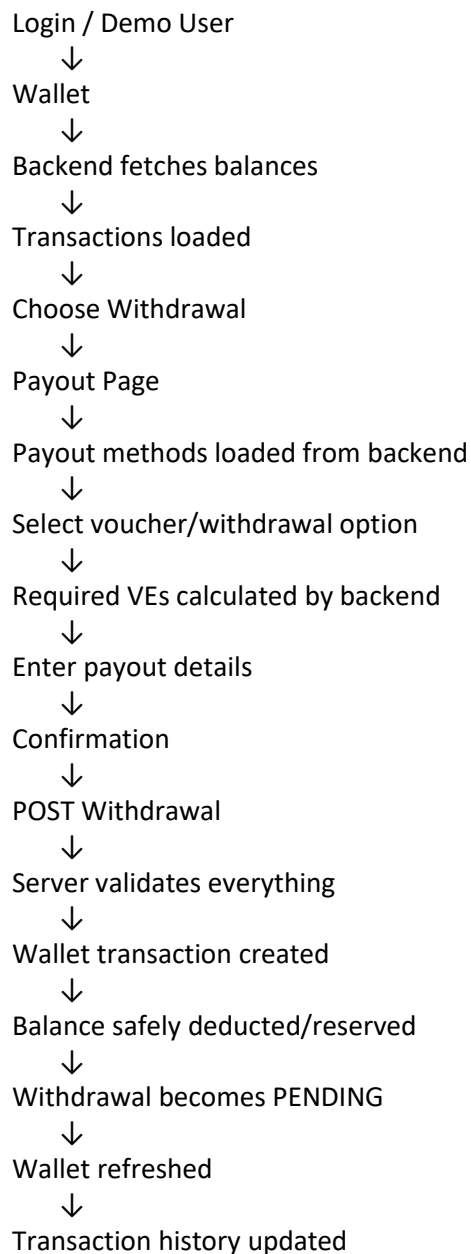
By **10 September 2026, 5:00 PM IST**, submit:

Required

- GitHub repository
 - Live frontend link
 - Live backend/API link
 - README.md
 - API documentation
 - Database/model documentation
 - Postman collection
 - Test cases/results
 - .env.example
 - Architecture explanation
 - Short explanation/video demonstrating the wallet and withdrawal flow
-

48. Final Demonstration

The final demonstration should show this complete flow:



49. Most Important Rule of This Task

The frontend is only the interface. The backend owns the money logic.

If someone opens DevTools and changes:

amount = 100

to:

amount = 100000

the backend must still reject an invalid request.

Likewise, changing:

VEs = 999999

in browser storage must **not** change the actual wallet.

VELoop's own documentation describes internal credits as platform-managed virtual units and states that withdrawals remain subject to verification, anti-fraud review, account standing, payout support and platform rules.

50. Final Challenge for the Intern

At the end of the project, answer this in your README:

“If VELoop Rewards grows from 1,000 users to 1,000,000 users, what changes would you make to this wallet architecture to maintain balance accuracy, transaction consistency, performance, security and withdrawal reliability?”

Your answer should discuss topics such as:

- database transactions
- atomic updates
- ledger architecture
- idempotency
- indexing
- queues
- caching
- rate limiting
- fraud detection
- audit logs
- reconciliation
- monitoring
- scalability

Submission Checklist

Before submitting, confirm:

- ☐ Wallet is completely backend-driven
- ☐ VEs are server-authoritative
- ☐ Other wallet currencies are server-authoritative
- ☐ Wallet transactions are stored
- ☐ Credits create ledger records
- ☐ Debits create ledger records
- ☐ Withdrawal requests are stored
- ☐ Payout options come from backend
- ☐ Payout values are not trusted from frontend
- ☐ Insufficient balance is rejected
- ☐ Duplicate withdrawal is prevented
- ☐ Concurrent withdrawal is handled safely
- ☐ Users cannot access another user's wallet
- ☐ Authentication is implemented
- ☐ Sensitive APIs are protected
- ☐ Validation is implemented
- ☐ Rate limiting is considered
- ☐ Audit logging is implemented
- ☐ Demo frontend is connected to backend
- ☐ /wallet flow works
- ☐ Payout flow works
- ☐ README is complete
- ☐ API documentation is complete
- ☐ Postman collection is included
- ☐ .env is not committed
- ☐ .env.example is included
- ☐ GitHub repository is submitted
- ☐ Live demo is submitted

- ☐ Test cases are documented

Timeline

Milestone	Date
Task Assigned	Today when its assigned
Duration	20 – 24 Days from its assigned date
Submission Time	5:00 PM IST

Note: The live VELoop Wallet/Payout pages should be treated as the UI and functional reference. The intern should not copy production code or receive production credentials. The objective is to independently build a clean, secure backend implementation that can later be integrated into VELoop Rewards. The platform's current withdrawal documentation also notes a ₹10 minimum withdrawal and that processing can normally be within 24 hours, with up to 72 hours in review cases.